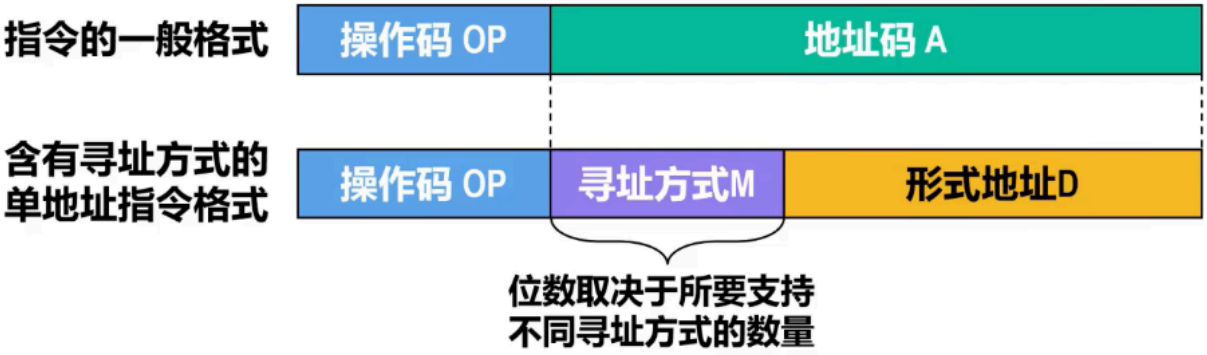
一、更详细的指令格式

■ 不同指令可能采用不同的操作数寻址方式来获取操作数。

□ 可将指令格式中的地址码字段划分为以下两部分：

◇ 寻址方式（寻址特征）字段M　　◇ 形式地址字段D



操作数寻址的过程，就是把寻址方式M和形式地址D的不同组合转换成有效地址的过程。

二、操作数寻址方式

2.1 立即寻址

操作数寻址方式
- 立即寻址
- 直接寻址
- 寄存器寻址
- 间接寻址
- 寄存器间接寻址
- 相对寻址
- 变址寻址
- 基址寻址
- 堆栈寻址
- 其他寻址



- 立即寻址方式下，操作数与指令是一起存放在主存的。
  - □ 取指令时，操作数随指令一起被读取并送到CPU内的指令寄存器IR中。
  - □ 指令执行时，直接从指令寄存器IR中获取操作数，无需访问其他存储单元。
- 立即寻址的特点如下：
  - □ 取操作数快。
  - □ 形式地址D的位宽有限，限制了操作数的表示范围。



- 立即寻址一般用于变量赋初值。

Intel x86中采用立即寻址的指令举例：

```
MOV  EAX, 5132408H     ；将十六进制立即数5132408送入32位通用寄存器EAX
```

2.2 直接寻址

操作数寻址方式
- 立即寻址
- **直接寻址**
- 寄存器寻址
- 间接寻址
- 寄存器间接寻址
- 相对寻址
- 变址寻址
- 基址寻址
- 堆栈寻址
- 其他寻址



- 直接寻址的**特点**如下：
  - 操作数的地址直观，无需专门计算操作数的地址。
  - 形式地址D的位宽有限，限制了操作数的寻址范围。
  - 操作数的地址位于指令中，因此不方便修改。

Intel x86中采用直接寻址的指令举例：

```
MOV  EAX, dword ptr [5132408H]  ; 从主存地址5132408H处；
                                ; 读取一个32位的操作数，
                                ; 并将该操作数送入32位
                                ; 通用寄存器EAX
```

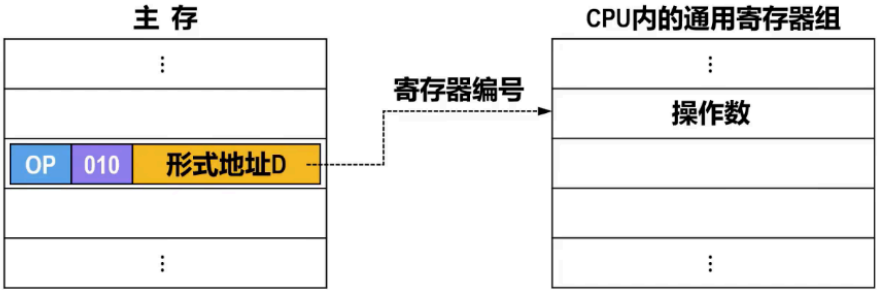


2.3 寄存器寻址

操作数寻址方式
- 立即寻址
- 直接寻址
- **寄存器寻址**
- 间接寻址
- 寄存器间接寻址
- 相对寻址
- 变址寻址
- 基址寻址
- 堆栈寻址
- 其他寻址



- 寄存器寻址的**特点**如下：
  - 获取操作数不需要访问主存（从寄存器获取），指令执行速度快。
  - 由于通用寄存器的数量少，因此所需的形式地址D的位数就少，有利于缩短指令字长，节省存储空间。



缺点及操作举例

- 由于CPU内的寄存器数量有限，因此寄存器寻址不能为大量的操作数提供存储服务。

Intel x86中采用寄存器寻址的指令举例：

```
MOV  EAX, EBX     ；将32位通用寄存器EBX中的内容送入32位通用寄存器EAX
```

2.4 间接寻址





- 间接寻址方式下，操作数与指令是分开存放在主存中的。
  - 指令中的形式地址D，给出的不是操作数的有效地址，而是操作数的间接地址，间接地址所指向的主存单元中的内容才是操作数的有效地址。
  - 取指令时，操作数不会随指令一起被读取并送到CPU内的指令寄存器IR中。
  - 指令执行时，需要访问两次主存以获取操作数。

右图给出的是单级间接寻址的示意图，
还可有多重间接寻址



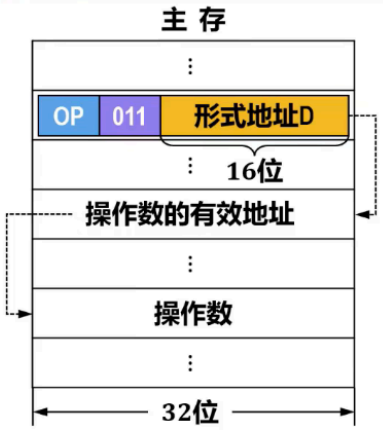
间接寻址的好处：==扩大寻址范围==



对比



为什么设计出这么麻烦的操作数寻址方式?



形式地址D（操作数的间接地址）的寻址范围为$2^{16} = 64k$

操作数（有效地址）的寻址范围为$2^{32} = 4G$

特点及操作举例

- 间接寻址的特点如下：
  - 相较于直接寻址，能用较短的形式地址D访问较大的主存空间。
  - 相较于直接寻址，当需要修改操作数地址时不再需要改变指令中的形式地址D，只需要修改形式地址D指向的主存单元（存有操作数地址）即可。
  - 指令执行时，需要访问两次主存以获取操作数，降低了指令的执行速度。

某计算机架构中采用间接寻址的指令举例：

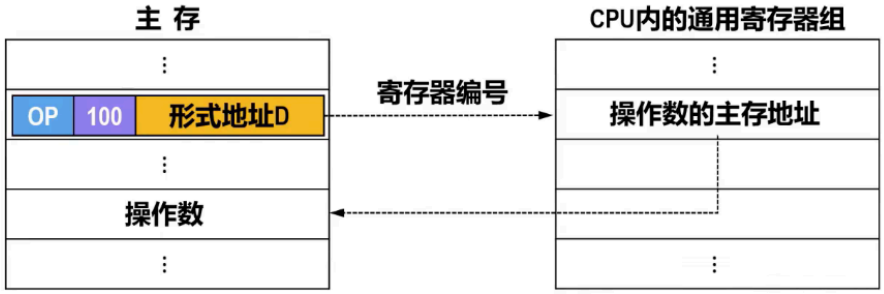
```
MOV  EAX, @5132408H     ;从主存地址5132408H所指向的存储单元中取出操作数的
                        ;有效地址，从该地址所指向的存储单元中取出操作数，
                        ;最后送入32位寄存器EAX;
```

2.5 寄存器间接寻址

操作数寻址方式
- 立即寻址
- 直接寻址
- 寄存器寻址
- 间接寻址
- **寄存器间接寻址**
- 相对寻址
- 变址寻址
- 基址寻址
- 堆栈寻址
- 其他寻址



- 指令中的形式地址D，给出的不是操作数的有效地址，而是通用寄存器的编号，而该编号的通用寄存器，其内容就是操作数的主存地址。
- 取指令时，操作数不会随指令一起被读取并送到CPU内的指令寄存器IR中。
- 指令执行时，需要访问一次主存以获取操作数。



特点及操作举例

- 寄存器间接寻址的特点如下：
  - 具有间接寻址的优点（扩充了操作数的寻址范围，提高了编程灵活性），还比间接寻址少访问一次主存。
  - 与寄存器寻址的缺点类似，由于CPU内的寄存器数量有限，因此寄存器间接寻址不能为大量的操作数提供存储其主存地址的服务。

Intel x86中采用寄存器间接寻址的指令举例：

```
MOV  EAX, [ EBX ]  ；将32位通用寄存器EBX中存储的主存地址所指向主存单元中的内容，
                   ；送入32位通用寄存器EAX
```

2.6 相对寻址





- 相对寻址方式下，操作数与指令是分开存放在主存中的。
  - 指令中的形式地址D，是需要给程序计数器PC中的内容加上的偏移量，可正可负（用补码表示），这样就可以形成操作数的主存地址或转移目标地址。

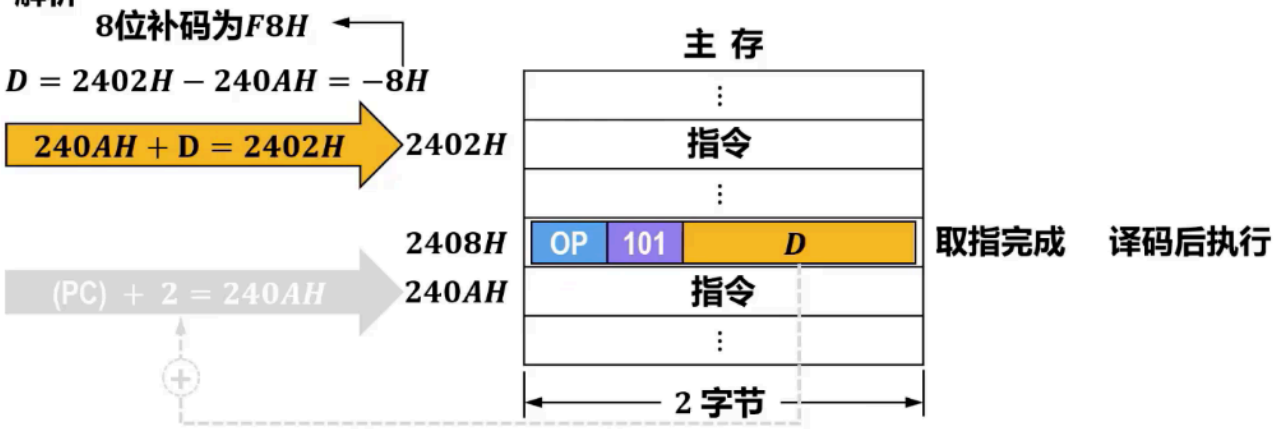


举例



【习题】假设某计算机采用定长指令系统，指令字长为16位，其中形式地址D字段为8位，主存按字节编址。若当前指令的主存地址为$2408H$，采用相对寻址，而转移后的的主存地址为$2402H$，则该相对寻址转移指令的形式地址D字段的值为多少？

解析



特点

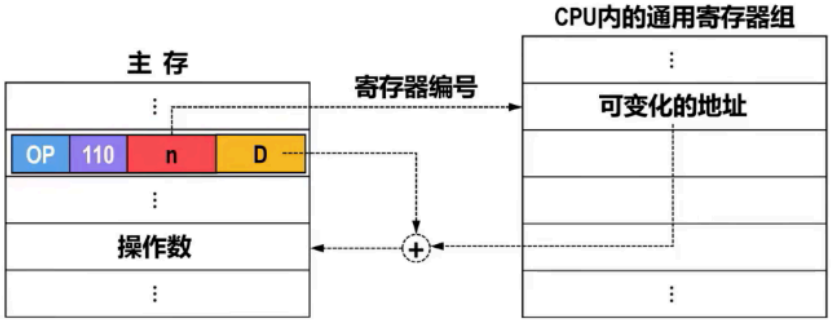


- 相对寻址方式下，操作数与指令是分开存放在主存中的。
  - 指令中的形式地址D，是需要给程序计数器PC中的内容加上的偏移量，可正可负（用补码表示），这样就可以形成操作数的主存地址或转移目标地址。
  - 取指令时，操作数不会随指令一起被读取并送到CPU内的指令寄存器IR中。
  - 指令译码或执行阶段，会计算操作数的主存地址或转移目标地址。
- 相对寻址的特点和用途如下：
  - 程序设计时只需给定指令与操作数之间的相对距离，而不必关心操作数在主存中的绝对地址，可用于实现共享代码的浮动。
  - 实现程序跳转执行。

2.7 变址寻址



- 变址寻址方式下，操作数与指令是分开存放在主存中的。
  - 指令中的寄存器编号n，指明使用CPU内部哪个寄存器（通用寄存器或专用寄存器），来存放变化的地址，该寄存器称为变址寄存器，其内部存放的地址与形式地址D相加，即可得到操作数的主存地址。
  - 取指令时，操作数不会随指令一起被读取并送到CPU内的指令寄存器IR中。
  - 指令执行时，需要访问一次主存以获取操作数。



特点

- 变址寻址的特点和用途如下：
  - 变址寄存器n的内容可变，而形式地址D的值设定后在指令执行过程中将保持不变。变址寄存器n的位数大于形式地址D的位数，扩大了寻址范围（可表示整个存储空间）。
  - 主要应用于对数组元素的访问。将数组的首地址赋值给指令中的形式地址D，使变址寄存器n的值按顺序变化，就可以对数组中的各元素进行相同的操作。

Intel x86中采用变址寻址的指令举例：

```
MOV  EAX, [ ESI + 32 ]  ；将32位通用寄存器ESI中的值加上偏移量32来形成主存地址，
                        ；再将主存地址所指向主存单元中的内容送入32位通用寄存器EAX
```

2.8 基址寻址

操作数寻址方式：立即寻址 / 直接寻址 / 寄存器寻址 / 间接寻址 / 寄存器间接寻址 / 相对寻址 / 变址寻址 / **基址寻址** / 堆栈寻址 / 其他寻址



面向用户，主要用于处理程序循环问题（例如对数组元素的访问），程序执行过程中，用户可以改变变址寄存器的内容来实现指令或操作数的寻址。

变址寻址	变址寄存器内容可变	值保持不变

基址寻址	基址寄存器内容不变	值可变

面向系统，主要用于程序的重定位。

- 对于多道程序设计，用户编程使用的是虚拟地址，当系统管理程序将用户程序载入主存时，会为每个用户程序分配一个基地址。
- 当某个用户程序运行时，分配给其的基地址被载入基地址寄存器，通过基址寻址方式实现虚拟地址到物理地址的转换。用户通过改变指令中的形式地址D来实现指令或操作数的寻址，而基址寄存器的内容只能由操作系统或系统管理程序通过特权指令设置，对用户透明。

2.9 堆栈寻址

操作数寻址方式：立即寻址 / 直接寻址 / 寄存器寻址 / 间接寻址 / 寄存器间接寻址 / 相对寻址 / 变址寻址 / 基址寻址 / **堆栈寻址** / 其他寻址

- 对存放在堆栈中的操作数进行寻址的方法称为堆栈寻址。
- 堆栈一般分为以下两种：
  - 寄存器堆栈
    - ✓ 在专用寄存器组中划分出来的一块、按“后进先出”（LIFO）规则管理的存储区，也称为硬堆栈。
    - ✓ 成本较高，不适合做大容量堆栈，较少使用。
  - 存储器堆栈
    - ✓ 在主存空间中划分出来的一块、按“后进先出”（LIFO）规则管理的存储区，也称为软堆栈。
    - ✓ 目前计算机普遍采用。
    - ✓ 由于存储器堆栈是在主存中划分出来的一块特定存储区，而主存基于地址进行访问，因此，需要使用一个特定的寄存器来记录堆栈中存储单元的地址，该寄存器称为堆栈指针（SP）。

举例

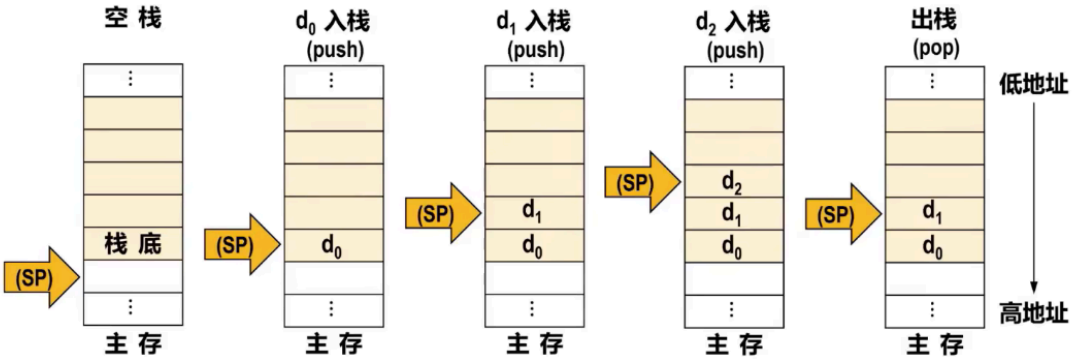
操作数寻址方式
- 立即寻址
- 直接寻址
- 寄存器寻址
- 间接寻址
- 寄存器间接寻址
- 相对寻址
- 变址寻址
- 基址寻址
- 堆栈寻址
- 其他寻址

□ 存储器堆栈

✓ 在主存空间中划分出来的一块、按“后进先出”（LIFO）规则管理的存储区，也称为软堆栈。

✓ 由于存储器堆栈是在主存中划分出来的一块特定存储区，而主存基于地址进行访问，因此，需要使用一个特定的寄存器来记录堆栈中存储单元的地址，该寄存器称为堆栈指针（SP）。

【举例】某计算机中，堆栈指针SP指向栈顶单元，栈向下（即低地址）增长，主存按字节编址，栈操作以字节为单位。



2.10 复合寻址

操作数寻址方式
- 立即寻址
- 直接寻址
- 寄存器寻址
- 间接寻址
- 寄存器间接寻址
- 相对寻址
- 变址寻址
- 基址寻址
- 堆栈寻址
- 其他寻址

- 将之前介绍的一些寻址方式进行组合，得到复合寻址方式。
- 复合寻址方式主要用于复杂指令集计算机CISC。

□ 相对寻址 + 间接寻址

❶ (PC) + D　相对寻址，得到存放操作数有效地址的主存单元的地址

❷ ((PC) + D)　间接寻址，得到操作数的有效地址

□ 变址寻址 + 间接寻址

❶ (变址寄存器) + D　变址寻址，得到存放操作数有效地址的主存单元的地址

❷ ((变址寄存器) + D)　间接寻址，得到操作数的有效地址

□ 间接寻址 + 变址寻址

❶ (D)　间接寻址，得到用于变址寻址的偏移量

❷ (变址寄存器) + (D)　变址寻址，得到操作数的有效地址

注意：

- 不同指令系统，可能采用了我们所介绍的寻址方式中的一部分。
- 我们所介绍的操作数寻址方式，都是单地址指令。对于多地址指令，每个地址字段的作用不同，因此每个地址字段都可能有各自的寻址方式（寻址特征）字段。

三、操作数寻址方式总结

指令格式 

	操作数 寻址方式	操作数的有效地址（主存地址）EA 程序计数器PC，形式地址D，通用寄存器N， 变址寄存器I，基址寄存器B	获取操作数需要的 访存次数 （不包括取指令）
	立即寻址	D为操作数本身	0
	直接寻址	$EA=D$	1
	寄存器寻址	$EA=N$	0
	（单级）间接寻址	$EA=(D)$	2
	寄存器间接寻址	$EA=(N)$	1
偏移寻址	相对寻址	$EA=(PC)+D$	1
偏移寻址	变址寻址	$EA=(I)+D$	1
偏移寻址	基址寻址	$EA=(B)+D$	1
复合寻址	相对寻址+间接寻址	$EA=((PC)+D)$	2
复合寻址	变址寻址+间接寻址	$EA=((I)+D)$	2
复合寻址	间接寻址+变址寻址	$EA=(I)+(D)$	2